

NAME: SHIVATHMIKA V

REG NO: 192521028

CSA1204 CO3AT1

1. Register Transfer Operations and Basic ALU Operations

Objective: Design and simulate a small register-transfer datapath that moves operands between multiple registers and performs addition, subtraction, AND, and OR operations using an ALU.

REGISTER TRANSFER + ALU DATA PATH

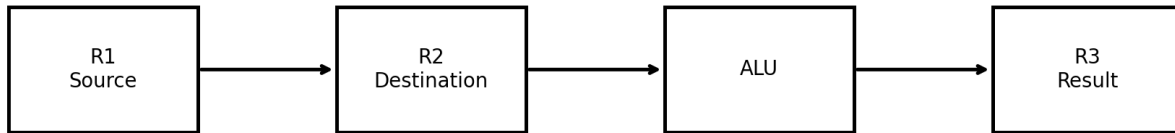


Figure 1. Conceptual register-transfer and ALU datapath.

1.1 System Design

The proposed datapath contains four registers (R1, R2, R3, and R4), a common internal transfer path, multiplexing/control logic, and an ALU. R1 and R2 provide operands to the ALU. The ALU produces the selected arithmetic or logic result, which is written into R3. R4 is used as an additional temporary register to demonstrate another transfer stage.

Component	Function
R1, R2	Store input operands
R3	Stores ALU output
R4	Temporary register for intermediate data
ALU	Performs ADD, SUB, AND and OR
Control unit	Generates register-load and ALU-select signals
Internal bus	Carries data between registers and ALU input/output paths

1.2 Register Transfer Notation

A register transfer is represented as $R2 \leftarrow R1$, meaning that the contents of R1 are copied into R2 on the active clock edge. For an ALU operation, the notation $R3 \leftarrow R1 + R2$ indicates that the ALU receives R1 and R2 and writes their sum into R3.

1. Load operand A into R1 and operand B into R2.
2. Transfer R1 to the ALU A-input and R2 to the ALU B-input.
3. Select the required ALU operation through the control signal.
4. Write the ALU output into R3 at the next active clock edge.
5. Optionally transfer R3 to R4 for temporary storage or further processing.

1.3 Example Simulation

Assume $R1 = 12$ and $R2 = 5$. The expected results are: ADD = 17, SUB = 7, AND = 4, and OR = 13. The exact bit pattern depends on the selected register width; an 8-bit datapath is sufficient for this experiment.

Operation	Control selection	Register-level action	Expected R3
Addition	ALU = ADD	$R3 \leftarrow R1 + R2$	17

Subtraction	ALU = SUB	$R3 \leftarrow R1 - R2$	7
AND	ALU = AND	$R3 \leftarrow R1 \text{ AND } R2$	4
OR	ALU = OR	$R3 \leftarrow R1 \text{ OR } R2$	13

1.4 Analysis

The experiment demonstrates that the ALU is the computational center of the datapath while registers provide fast temporary storage. Separating operand storage, operation selection, and result storage makes the execution sequence easy to control. A common transfer path reduces wiring complexity, but only the resources enabled by the control unit can participate in a particular clock cycle.

1.5 Result

The proposed model successfully demonstrates register-to-register transfers and the four required ALU functions. The sequence shows how control signals coordinate data movement and computation, satisfying the required analysis of the ALU's role.

2. Single-Bus and Multi-Bus Computer Organization

Objective: Model data transfers among registers, memory, I/O and the ALU using single-bus and multi-bus organizations, then compare their transfer efficiency and bottlenecks.

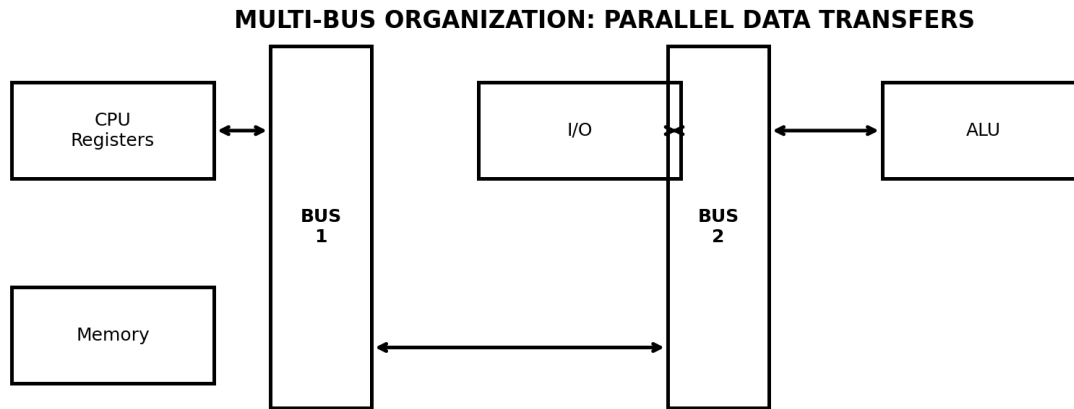


Figure 2. Conceptual multi-bus organization with parallel transfer paths.

2.1 Single-Bus Organization

In a single-bus organization, major processor registers and functional units share one common bus. Only one source can place data on the bus at a time, so a multi-step operation may require several clock cycles. For example, an ALU instruction needing two operands may first move one operand to a temporary register and then move the second operand before the ALU can complete the operation.

2.2 Multi-Bus Organization

A multi-bus organization provides two or more independent data paths. A typical three-bus datapath can supply two operands to the ALU simultaneously and use a third path for the result. This reduces the number of sequential transfers and allows greater overlap between independent data movements.

Feature	Single bus	Multi-bus
Number of transfer paths	One shared path	Two or more independent paths
Operand movement	Mostly sequential	Can be parallel
Hardware cost	Lower	Higher
Control complexity	Lower	Higher
Bottleneck risk	Higher	Lower
ALU utilization	May be limited by transfers	Generally improved

2.3 Example Transfer Sequence

For an operation $R3 \leftarrow R1 + R2$, a single-bus design may require multiple cycles to move $R1$ and $R2$ through temporary registers before addition. In a two-source multi-bus design, $R1$ and $R2$ can be placed on separate source buses at the same time, allowing the ALU to receive both operands in one transfer phase and write the result through the destination path.

2.4 Performance Comparison

Metric	Single-bus	Multi-bus	Interpretation
Operand transfers	Sequential	Parallel	Multi-bus reduces

			transfer dependency
Potential cycles per ALU instruction	Higher	Lower	Fewer intermediate moves
Throughput	Lower under heavy traffic	Higher	More operations can progress
Bus contention	Frequent	Reduced	Independent paths share less traffic
Implementation cost	Lower	Higher	Extra buses and control logic are required

2.5 Analysis

The main advantage of multiple buses is parallel data transfer. The improvement becomes especially important when several operands or functional units must communicate frequently. However, adding buses increases hardware area, wiring, switching activity, and control complexity. Therefore, the appropriate organization is a trade-off between performance and implementation cost.

2.6 Result

The multi-bus organization provides better transfer parallelism and reduces the bottleneck caused by a single shared path. The single-bus structure remains attractive for simpler and lower-cost designs, while multi-bus structures are better suited to higher-throughput processor datapaths.

3. Five-Stage Instruction Pipeline

Objective: Simulate a five-stage instruction pipeline consisting of Fetch, Decode, Execute, Memory and Write-back, and compare its performance with a non-pipelined processor.

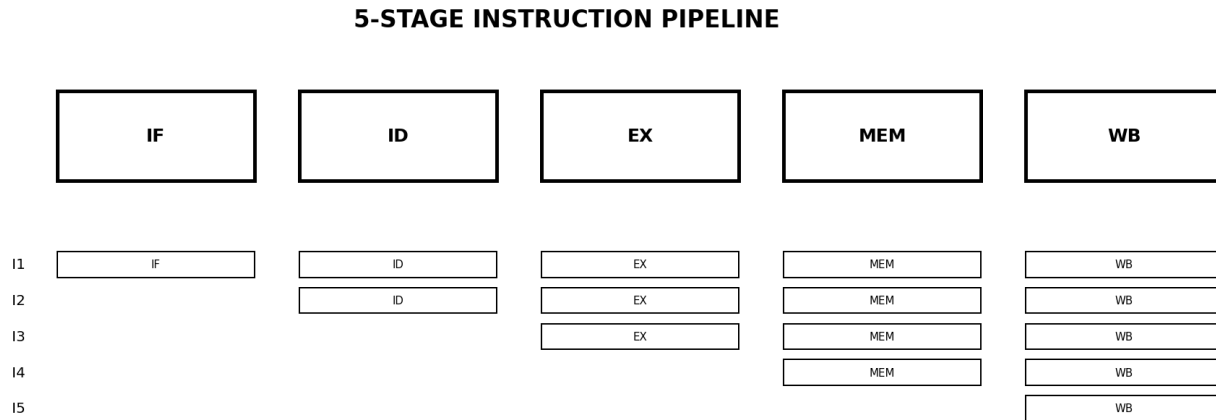


Figure 3. Five-stage pipeline timing concept.

3.1 Pipeline Stages

Stage	Name	Function
IF	Instruction Fetch	Fetch instruction from instruction memory/cache
ID	Instruction Decode	Decode opcode and read required registers
EX	Execute	Perform ALU operation or calculate an address
MEM	Memory	Read/write data memory when required
WB	Write-back	Write the final result into the destination register

3.2 Instruction Set for Simulation

Use the following independent instruction sequence for the basic timing comparison: I1: ADD R1,R2,R3; I2: SUB R4,R5,R6; I3: AND R7,R8,R9; I4: OR R10,R11,R12; I5: ADD R13,R14,R15. The instructions are chosen to minimize data dependencies so that the ideal pipeline effect can be observed first.

3.3 Timing Table

Cycle	I1	I2	I3	I4	I5
1	IF				
2	ID	IF			
3	EX	ID	IF		
4	MEM	EX	ID	IF	
5	WB	MEM	EX	ID	IF
6		WB	MEM	EX	ID
7			WB	MEM	EX
8				WB	MEM

9					WB
---	--	--	--	--	----

3.4 Performance Calculation

For a five-stage pipeline executing n instructions under ideal conditions, the approximate completion time is $k + n - 1$ cycles, where $k = 5$. For five instructions, the pipelined completion time is $5 + 5 - 1 = 9$ cycles. A non-pipelined processor that takes five stage-times per instruction would require $5 \times 5 = 25$ stage-times. Therefore, the ideal speedup for this small sequence is $25/9 \approx 2.78\times$.

For a large instruction stream, the ideal speedup approaches the number of pipeline stages, but real processors achieve less because of data, control and structural hazards, unequal stage delays, pipeline fill/drain overhead, cache misses and other stalls.

3.5 Throughput

After the pipeline is filled, one completed instruction can ideally leave the pipeline every clock cycle. Thus, throughput approaches one instruction per cycle ($IPC \approx 1$ for a single-issue ideal pipeline). The non-pipelined design completes an instruction only after all five stages have been performed sequentially.

3.6 Result and Analysis

The simulation shows that pipelining improves instruction throughput by overlapping different stages of multiple instructions. Although the latency of one instruction is still approximately five stages, the overall completion rate increases substantially because the hardware is kept busy with different instructions at the same time.

4. Pipeline Hazards and Their Resolution

Objective: Develop a pipeline scenario containing data, control and structural hazards, observe their impact, and apply stalling, forwarding and branch prediction to improve execution.

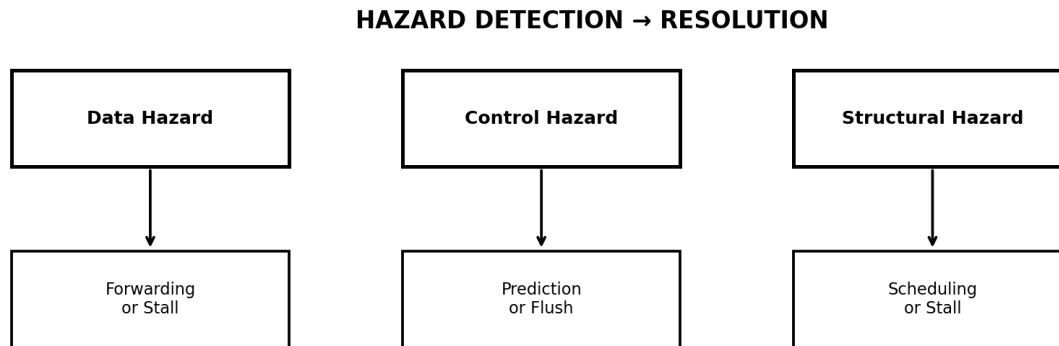


Figure 4. Hazard types and common resolution techniques.

4.1 Data Hazard

A data hazard occurs when an instruction depends on a result that a previous instruction has not yet written back. For example: I1: ADD R1,R2,R3 followed by I2: SUB R4,R1,R5. I2 needs the new R1 value, so using the old register value would produce an incorrect result.

Resolution: Data forwarding sends the result from a later pipeline stage directly to the dependent instruction's input instead of waiting for normal write-back. If forwarding cannot provide the value in time, the control unit inserts a stall (bubble).

4.2 Control Hazard

A control hazard occurs when the next instruction depends on the outcome of a branch. Until the branch direction and target are known, the processor may not know which instruction should be fetched next.

Resolution: Branch prediction guesses the likely path and allows fetching to continue. If the prediction is correct, the pipeline avoids a long delay. If it is wrong, incorrectly fetched instructions are flushed and the correct target path is restarted.

4.3 Structural Hazard

A structural hazard occurs when two pipeline stages require the same hardware resource at the same time. A simple example is a design with one memory that must serve both an instruction fetch and a data-memory access during the same cycle.

Resolution: The processor can add or separate resources, schedule operations to avoid conflicts, or stall one instruction until the shared resource becomes available.

Hazard	Cause	Main technique	Effect
Data	Instruction depends on an unfinished result	Forwarding / stall	Avoids incorrect operand use
Control	Branch changes instruction flow	Prediction / flush	Reduces branch delay
Structural	Two stages need same	Duplicate resource / stall	Removes resource

	hardware		conflict
--	----------	--	----------

4.4 Combined Scenario

Consider: I1: LOAD R1,[R2]; I2: ADD R3,R1,R4; I3: BEQ R3,R0,TARGET; I4: SUB R5,R6,R7. I2 has a load-use dependency on I1, I3 depends on the result of I2, and the branch creates a control hazard. If instruction and data access share one memory resource, I1 can also introduce a structural conflict. A practical pipeline controller detects each condition and applies forwarding where possible, inserts a short stall when required, and uses branch prediction to reduce control delay.

4.5 Analysis

Hazards prevent an ideal pipeline from completing one instruction every cycle. Forwarding usually provides the most direct solution for short data dependencies; stalls are a safe fallback when a value is not yet available. Branch prediction improves average performance by keeping the front end active, while structural hazards are best reduced through suitable hardware organization or careful scheduling.

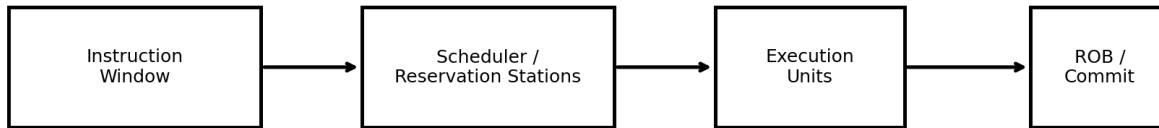
4.6 Result

The designed scenario demonstrates all three major pipeline hazards and shows how forwarding, stalling and branch prediction preserve correctness while reducing the performance loss caused by hazards.

5. Superscalar, Out-of-Order, Speculative Execution and SMT

Objective: Analyze a processor capable of executing multiple instructions per cycle, using out-of-order and speculative execution, and extend the analysis to hyper-threading/simultaneous multithreading (SMT).

SUPERSCALAR + OUT-OF-ORDER + SPECULATIVE EXECUTION



SMT: multiple threads share execution resources while the processor issues independent instructions.

Figure 5. Simplified superscalar execution flow.

5.1 Superscalar Execution

A superscalar processor contains multiple execution resources and can issue more than one independent instruction in a clock cycle. For example, a two-issue processor may dispatch two independent instructions in one cycle when the required functional units and operands are available.

5.2 Out-of-Order Execution

Out-of-order execution allows ready instructions to execute before older instructions that are temporarily waiting for operands or resources. The processor tracks dependencies and resource availability using structures such as reservation stations or a scheduling mechanism. Results are later committed in program order so that the architectural state remains correct.

5.3 Speculative Execution

Speculative execution allows the processor to execute instructions before it is certain that they belong to the final control-flow path. Branch prediction supplies a likely path. If the prediction is correct, useful work has already been performed; if incorrect, speculative results are discarded and execution continues from the correct path.

5.4 Simultaneous Multithreading (SMT)

SMT allows instructions from multiple software threads to share a single physical processor core's execution resources. When one thread is stalled, instructions from another thread may use otherwise idle resources. This can improve utilization and overall throughput, although the threads compete for shared resources such as execution units, caches and issue bandwidth.

Technique	Main idea	Performance benefit	Limitation
Superscalar	Issue multiple independent instructions	Higher instruction throughput	Needs more hardware/resources
Out-of-order	Execute ready instructions before stalled older ones	Hides some latency	Complex dependency tracking
Speculative	Execute predicted	Reduces branch-related	Wrong predictions

	control-flow path	idle time	require recovery
SMT	Issue instructions from multiple threads	Improves resource utilization	Threads share and compete for resources

5.5 Example Throughput Comparison

Assume an ideal single-issue pipeline can retire about one instruction per cycle. An ideal two-issue superscalar processor can retire up to two instructions per cycle when enough independent work exists. If a workload contains many dependencies, cache misses or branch mispredictions, the measured throughput will be lower than the theoretical maximum.

Processor model	Ideal issue width	Relative theoretical throughput
Single-issue pipeline	1 instruction/cycle	1×
2-way superscalar	2 instructions/cycle	2×
2-way superscalar + SMT (2 threads)	Up to 2 instructions/cycle shared across threads	Up to 2× core issue capacity, with better utilization when one thread stalls

5.6 Resource Utilization Analysis

Superscalar and out-of-order techniques attempt to keep execution units busy by finding instruction-level parallelism. SMT extends this idea across threads. The final performance depends on workload characteristics: independent instructions increase utilization, while heavy dependencies, branch mispredictions and memory stalls reduce the achievable throughput.

5.7 Result

The analysis demonstrates that superscalar issue increases the number of instructions that can be processed in parallel, out-of-order execution hides some latency by selecting ready instructions, speculative execution reduces the cost of likely control-flow paths, and SMT improves utilization by sharing a core among multiple threads.

Overall Conclusion

The five experiments collectively demonstrate how processor organization affects performance. Register transfers and ALU control form the basic datapath operations; multi-bus structures improve parallel data movement; pipelining overlaps instruction stages; hazard-resolution mechanisms preserve correctness while reducing stalls; and superscalar, out-of-order, speculative and SMT techniques increase instruction-level and thread-level parallelism. The progression from simple datapaths to advanced execution techniques shows that higher performance is achieved by increasing useful parallelism while carefully managing hardware resources and dependencies.